

Examen de mi-semester SRCS Master 1 SAR

Jonathan Lejeune

Mars 2022

2 heures – **Tout document papier interdit**
Tout appareil électronique interdit

- Le barème est sur 21 points et est donné à titre indicatif. La note finale sera bornée à 20 points.
- Dans le code java demandé vous ne gérerez pas les imports
- Soyez synthétique et précis dans vos réponses
- Il vous est conseillé de lire attentivement les exercices entièrement avant de composer.
- Lorsque vous ne savez pas répondre à une question, il est préférable de s'abstenir que de répondre une absurdité.
- Vous trouverez à la fin du sujet, une annexe résumant les API à utiliser.

Exercice 1 – Questions de cours (5 points)

Question 1 1/2 point

Dans la classe `DataOutputStream`, quelle est la différence entre la méthode `void write(int);` et `void writeInt(int);`;

Question 2 1 point

Donner un avantage et un inconvénient du modèle client-serveur.

Question 3 1 point

Quelles sont les informations que l'on doit trouver dans la spécification d'un protocole applicatif?

Question 4 1 point

Dans quel cas la méthode `readObject` de la classe `ObjectInputStream` jette-t-elle une `ClassNotFoundException`?

Question 5 1 1/2 points

Soit A une classe non chargée. Quelles sont les étapes de traitements préalables effectuées par la JVM pour pouvoir instancier A en mémoire? Préciser pour chaque étape, son objectif.

Exercice 2 – Copie en profondeur (3,5 points)

Dans une classe `Util` on souhaite coder la méthode utilitaire statique suivante :

```
static <T> T deepCopy(T obj);
```

Cette méthode renvoie une copie profonde de l'objet `obj` passer en paramètre. La variable de type `T`, locale à la méthode, désigne le type de la variable qui référence l'objet à copier. Pour implanter cette méthode, on propose d'utiliser le mécanisme de la sérialisation d'objet offert par la JVM.

Question 1 1/2 point

Quel problème de la copie profonde, la sérialisation résout-elle ?

Question 2 1/2 point

Pourquoi cette solution n'est-elle pas applicable à tous les objets ?

Question 3 1/2 point

Dans quel cas est-il possible que le résultat ne soit pas une copie parfaite du paramètre ?

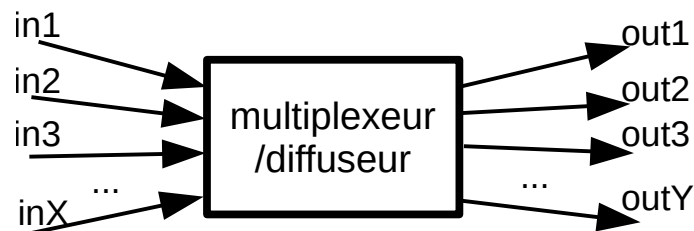
Question 4 2 points

Donner le code de cette méthode en prenant comme support un buffer mémoire

(cf. `ByteArrayInputStream` et `ByteArrayOutputStream` qui permettent respectivement de lire et d'écrire dans un tableau d'octets).

Exercice 3 – Multiples flux d'objets (12,5 points)

Le but de cet exercice est de programmer une brique logicielle permettant de multiplexer un ensemble de flux d'objets d'entrée pour faire une diffusion sur un ensemble de flux d'objets de sortie.



Ainsi tout objet reçu sur n'importe quel flux d'entrée doit être émis sur tous les flux de sortie. On notera que le nombre de flux (sortie et entrée) n'est pas défini statiquement mais dynamiquement, c'est-à-dire que l'on peut ajouter au cours de l'exécution des flux d'entrée ou de sortie. Pour simplifier, on ne gèrera pas la suppression de flux.

Pour concevoir ce composant, nous allons procéder en trois temps :

- Une première étape pour la gestion des flux de sortie
- Une deuxième étape pour la gestion des flux d'entrée
- Une troisième étape pour assembler le tout.

Question 1 3 points

Donner le code d'une classe `ObjectStreamBroadcast` qui offre les méthodes publiques suivantes :

- `void addOutput(ObjectOutputStream out)` permet d'ajouter un nouveau flux de sortie sur lequel diffuser.
- `void writeObject(Object o)` : qui diffuse à l'ensemble des flux de sortie enregistrés un objet donné. Vous devez gérer le fait que si une erreur survient dans l'écriture d'un flux, ceci ne remette pas en cause l'écriture dans les autres flux.
- `void close()` : qui ferme tous les flux enregistrés. Tout appel à `writeObject()` ou `addOutput()` après un appel à `close()` doit jeter une exception runtime `AlreadyClosedException` (classe d'exception que l'on supposera existante).

Vous devez gérer le fait que les appels à ces trois méthodes peuvent se faire dans un contexte multi-threadé et qu'il est donc nécessaire d'assurer la cohérence sur l'état de l'objet.

Question 2 5 points

Donner le code d'une classe `ObjectStreamMultiplex` qui offre les méthodes publiques suivantes :

- `void addInput(ObjectInputStream in)` permet d'ajouter un nouveau flux d'entrée sur lequel lire. Lors de l'ajout d'un nouveau flux, un nouveau fil d'exécution sera créé et dédié à la lecture d'objets sur le flux. Il se termine dès que la lecture jette une exception. Dans ce cas, le flux associé sera fermé. Les objets lus seront stockés dans un tampon.

- `Object readObject()` : qui retourne l'objet le plus ancien inséré dans le tampon par les fil d'exécutions lisant les flux entrants. L'appel est bloquant tant que le tampon est vide ou jusqu'à ce que l'on ait appelé la méthode `close()` décrite ci-après. Dans ce cas, on retourne la référence nulle. Si le fil d'exécution appelant cette méthode est interrompu pendant son blocage, on retournera également la référence nulle.
- `void close()` : qui ferme tous les flux entrants enregistrés. De la même manière que la question précédente, tout appel à `addInput` après un `close` jettera une `AlreadyClosedException`.

L'assemblage de notre multiplexeur/diffuseur consistera à programmer un serveur qui écoute sur deux ports TCP : un port TCP sera dédié aux clients souhaitant émettre des objets (i.e. les sources des inputs) tandis que l'autre sera dédié aux clients souhaitant recevoir des objets (i.e. les destinataires des outputs). Ainsi tout objet envoyé par un client émetteur sera reçu par tous les clients récepteurs.

Question 3 4 points

En utilisant les deux classes des questions précédentes, donner le code d'une classe `Serveur` qui offre :

- un constructeur prenant en paramètre les numéros des deux numéros de port et qui instancie deux sockets d'écoute. Si le déploiement d'une des deux socket échoue alors le constructeur jette une `IOException` et annulera le cas échéant le déploiement de l'autre socket.
- une méthode `start()` qui permet de démarrer le serveur et le rendre prêt à recevoir des requêtes sur les deux ports.
- une méthode `stop()` qui arrête le serveur. L'appel à cette méthode assure que toute ressource créée pendant le déploiement et l'exécution du serveur (sockets d'écoute, sockets de communication avec les clients, threads, ...) soient fermée et/ou détruit. Une connexion sur le port des inputs (respectivement sur le port des outputs) implique donc de rajouter un flux d'entrée (resp. de sortie) associé à la socket de communication sur la liste des inputs (resp. des outputs).

Question 4 1/2 point

Quel hypothèse devons-nous faire sur la JVM qui héberge une instance de la classe `Serveur` pour que ce mécanisme fonctionne ?

Java I/O classes

Abstract Classes	
Class	Main public fields
abstract class InputStream implements Closeable	• abstract int read() throws IOException • int read(byte b[]) throws IOException • int read(byte b[], int off, int len) throws IOException • void close() throws IOException
abstract class OutputStream implements Closeable, Flushable	• abstract void write(int b) throws IOException • void write(byte b[]) throws IOException • void write(byte b[], int off, int len) throws IOException • void flush() throws IOException • void close() throws IOException

Main InputStream sub-classes

Class	Main public fields
class FileInputStream extends InputStream	• public FileInputStream(String name) throws FileNotFoundException • public FileInputStream(File file) throws FileNotFoundException
class ByteArrayInputStream extends InputStream	• public ByteArrayInputStream(byte buf[]) • public ByteArrayInputStream(byte buf[], int offset, int length)
class BufferedInputStream extends FilterInputStream	• public BufferedInputStream(InputStream in)
class DataInputStream extends FilterInputStream implements DataInput	• public DataInputStream(InputStream in) • String readLine() throws IOException • double readDouble() throws IOException • float readFloat() throws IOException • String readUTF() throws IOException • long readLong() throws IOException • int readInt() throws IOException • char readChar() throws IOException • boolean readBoolean() throws IOException • byte readByte() throws IOException
class ObjectInputStream extends InputStream implements ObjectInput, ObjectStreamConstants	• public ObjectInputStream(InputStream in) throws IOException • methods of DataInputStream • final Object readObject() throws IOException, ClassNotFoundException
class PipedInputStream extends InputStream	• public PipedInputStream(PipedOutputStream src) throws IOException • public PipedInputStream() • void connect(PipedOutputStream src) throws IOException

Main OutputStream sub-classes

Class	Main public fields
class FileOutputStream extends OutputStream	• public FileOutputStream(String name) throws FileNotFoundException • public FileOutputStream(File file) throws FileNotFoundException
class ByteArrayOutputStream extends OutputStream	• public ByteArrayOutputStream() • public ByteArrayOutputStream(int size) • public byte[] toByteArray()
class BufferedOutputStream extends FilterOutputStream	• public BufferedOutputStream(OutputStream out)

class DataOutputStream extends FilterOutputStream implements DataOutput	• public DataOutputStream(OutputStream out) • void writeDouble(double v) throws IOException • void writeFloat(float v) throws IOException • void writeUTF(String str) throws IOException • void writeLong(long v) throws IOException • void writeInt(int v) throws IOException • void writeChar(int v) throws IOException • void writeBoolean(boolean v) throws IOException • void writeByte(int v) throws IOException • void writeBytes(String s) throws IOException
class ObjectOutputStream extends OutputStream implements ObjectOutput, ObjectOutputStreamConstants	• public ObjectOutputStream(OutputStream out) throws IOException • methods of DataOutputStream • void writeObject(Object obj) throws IOException
class PipedOutputStream extends OutputStream	• public PipedOutputStream(PipedInputStream snk) throws IOException • public PipedOutputStream() • void connect(PipedInputStream snk) throws IOException